



Spezifikationsdokument

in Phase 3 aktualisiert

Projekttitel: Network Architect

Kurs: Software Engineering (DLMCSPSE01_D)

Student: Johannes Öllerer

Matrikelnummer: 92211477

Datum: 01.August 2026



Inhaltsverzeichnis

1 Datenmodell.....	3
1.1 Geschäftsobjekte.....	3
1.2 Verwendete Beziehungstypen	4
1.3 Beziehungen der Geschäftsobjekte.....	4
1.4 UML-Klassendiagramm.....	5
2 Geschäftsprozesse	6
2.1 Übersicht der Prozessgruppen	6
2.2 Detaillierte Prozessbeschreibungen.....	6
2.2.1 Spielverwaltung.....	6
2.2.2 Gameplay-Kernprozesse.....	8
2.2.3 Progression und Feedback.....	8
2.2.4 Tutorial/Hilfesystem.....	9
2.3 UML-Aktivitätsdiagramm eines ausgewählten Prozesses	10
3 Geschäftsregeln.....	11
3.1 Budget- und Kostenregeln.....	11
3.2 Netzwerk-Topologie-Regeln.....	11
3.3 Node-Spezialisierungs-Regeln	11
3.4 Performance- und Redundanz-Bewertungsregeln.....	11
3.5 Spielablauf-Regeln	12
4 Systemschnittstellen.....	12
5 Benutzerschnittstellen.....	13
5.1 Hauptspielbildschirm	13
5.2 Fehlermeldungsdialog bei Regelverstoß.....	14
5.3 Levelabschluss-Dialog.....	14
6 Abbildungsverzeichnis.....	15

1 Datenmodell

1.1 Geschäftsobjekte

Node

Ein Knotenpunkt im Netzwerk-Raster, an dem Brücken ansetzen können. Jeder Node hat eine Position im Raster, einen Netzwerkgerätetyp (Client, Server, Switch, Route und Firewall) und eine maximale Anzahl erlaubter Brückenverbindungen. Während des Spiels wird die aktuelle Anzahl der Brückenverbindungen erfasst.

Bridge

Eine Verbindung zwischen zwei Nodes im Netzwerk. Bridges können parallel (bis zu 2) zwischen denselben Nodes existieren und haben unterschiedliche Verbindungstypen wie WLAN, 4G, 5G oder LAN-Kabel, die später die Performance beeinflussen.

Network

Neben der Speicherung der Nodes und Bridges übernimmt das Network auch die Topologie-Validierung. Es prüft, ob das Netzwerk den Spielregeln entspricht, beispielsweise ob alle Knoten korrekt verbunden sind, die maximale Anzahl Brücken nicht überschritten wird und ob Redundanzkriterien erfüllt sind.

Level

Ein Level ist eine vordefinierte Spielstufe mit fester Rastergröße und Startpositionen der Nodes. Jedes Level besitzt eine eindeutige Level-ID, über die es intern identifiziert werden kann. Neben der räumlichen Anordnung der Nodes legt das Level zudem den Schwierigkeitsgrad sowie eine Zielpunktzahl fest. Levels definieren damit die Rahmenbedingungen für ein einzelnes Puzzle, das der Spieler lösen muss.

GameSession

Eine GameSession repräsentiert eine einzelne Spielsitzung und enthält Benutzerdaten. Zur eindeutigen Zuordnung verfügt jede GameSession über eine Session-ID, unter der Spielfortschritt und benutzerspezifische Statistiken gespeichert werden. Sie hält Informationen zu Benutzername, erreichten Scores, Zeitmessungen und benutzerspezifischen Spielerstatistiken sowie das aktuell gespielte Level und das zugehörige Network. Die GameSession verwaltet damit den Momentanzustand des Spiels und ermöglicht das Speichern und Laden von Spielständen.

1.2 Verwendete Beziehungstypen

In dieser Ausarbeitung werden folgende Beziehungstypen verwendet:

- 1:1 Ein Objekt ist genau mit einem anderen Objekt verbunden.
- 1:* Ein Objekt ist mit mehreren Objekten verbunden.
- *.* Mehrere Objekte sind mit mehreren anderen Objekten verbunden.
- 1:2 Ein Objekt ist mit genau zwei anderen Objekten verbunden (z. B. Bridge verbindet genau zwei Nodes).

1.3 Beziehungen der Geschäftsobjekte

GameSession → Level

Beziehungstyp: 1:*

Eine GameSession umfasst mehrere Levels. Ein Spieler kann in einer Spielsession mehrere Levels nacheinander spielen.

Level → Network

Beziehungstyp: 1:1

Jedes Level hat genau ein aktuelles Network, das die vom Spieler erstellte Netzwerkstruktur repräsentiert.

Network → Nodes

Beziehungstyp: 1:*

Ein Network besteht aus mehreren Nodes, die die Knotenpunkte im Spielnetzwerk darstellen.

Network → Bridges

Beziehungstyp: 1:*

Ein Network besteht aus mehreren Bridges, die als Verbindungen zwischen Nodes dienen.

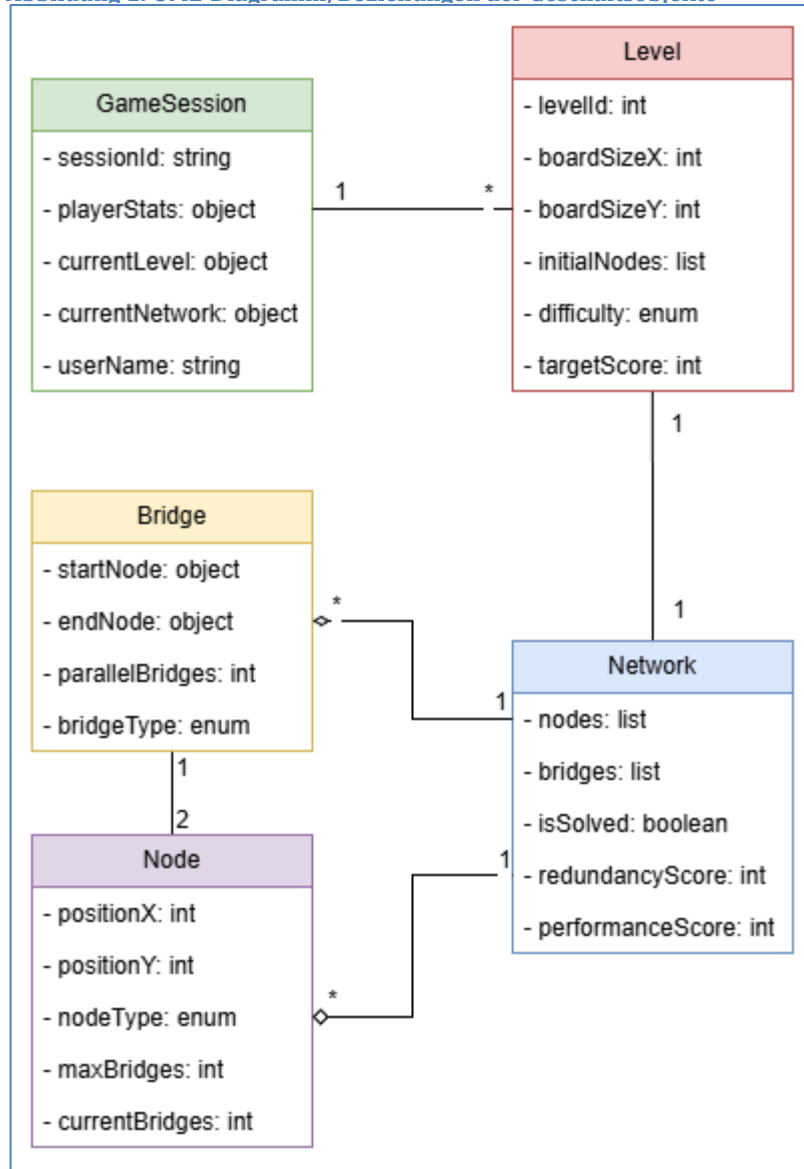
Bridge → Nodes

Beziehungstyp: 1:2

Jede Bridge verbindet genau zwei Nodes: einen Start- und einen Endknoten. Zwischen denselben Nodes können bis zu zwei parallele Bridges existieren.

1.4 UML-Klassendiagramm

Abbildung 1: UML-Diagramm, Beziehungen der Geschäftsobjekte



Quelle: erstellt mit draw.io

2 Geschäftsprozesse

2.1 Übersicht der Prozessgruppen

Die Geschäftsprozesse von Network Architect sind in fünf logische Gruppen unterteilt, die den gesamten Lebenszyklus einer Spielsession abbilden:

Spielverwaltung (Prozesse 1-5)

Umfasst App-Start, Benutzerauswahl, Level-Navigation und Spielstand-Verwaltung. Stellt die Grundlage für alle Spielaktivitäten bereit

Gameplay(Prozesse 6-8)

Zentrale Spielmechanik: Brücken platzieren mit Validierung nach Spielregeln sowie Lösungskontrolle für Level-Vervollständigung.

Progression und Feedback (Prozesse 9-11)

Verarbeitet Level-Abschluss, Score-Berechnung und persistenten Fortschritt mit Statistik-Speicherung.

Tutorial/Hilfesystem (Prozesse 12-13)

Bietet Einstiegshilfe für neue Spieler sowie strategische Hinweise während des Spiels.

2.2 Detaillierte Prozessbeschreibungen

2.2.1 Spielverwaltung

Prozess 1: Spielstart und Initialisierung

- | | |
|--------------------------------|---|
| • Beschreibung: | Die Anwendung wird gestartet. Es wird das Hauptmenü geladen, verfügbare Benutzerprofile werden angezeigt und die grundlegenden Spielressourcen initialisiert. |
| • Beteiligte Geschäftsobjekte: | GameSession |
| • Eingabe: | Keine (App-Start) |
| • Ausgabe: | Hauptmenü bereit |
| • Erfolgsbedingung: | Menü interaktiv |

Prozess 2: Benutzer-Auswahl

- Beschreibung: Gespeicherte Benutzerprofile werden angezeigt. Auswahl oder Neues erstellen.
- Beteiligte Geschäftsobjekte: GameSession
- Eingabe: Benutzer-ID oder Name
- Ausgabe: Aktive GameSession
- Erfolgsbedingung: Profil geladen

Prozess 3: Level-Auswahl und Navigation

- Beschreibung: Freigeschaltete Levels werden angezeigt. Spieler wählt Level aus.
- Beteiligte Geschäftsobjekte: GameSession; Level
- Eingabe: Level-ID
- Ausgabe: Level geladen
- Erfolgsbedingung: Gameplay bereit

Prozess 4: Spielstand laden/speichern

- Beschreibung: Spielzustand wird persistent gespeichert/geladen.
- Beteiligte Geschäftsobjekte: GameSession
- Eingabe: Session-ID
- Ausgabe: Spielstand persistent
- Erfolgsbedingung: Zustand gespeichert

Prozess 5: Level laden und initialisieren

- : Vordefinierte Level werden geladen und für Gameplay initialisiert (Nodes positioniert, leeres Network erzeugt).
- Beteiligte Geschäftsobjekte: Level, GameSession
- Eingabe: Level-ID
- Ausgabe: Spielbereites Level mit Node-Positionen und leerem Network
- Erfolgsbedingung: Gameplay startbereit

2.2.2 Gameplay-Kernprozesse

Prozess 6: Brücke platzieren/entfernen

- : Spieler platziert oder entfernt eine Bridge zwischen zwei Nodes.
- Beteiligte Geschäftsobjekte: Bridge, Node, Network
- Eingabe: Node1, Node2, Bridge-Typ
- Ausgabe: Aktualisiertes Network
- Erfolgsbedingung: Bridge geändert

Prozess 7: Verbindungsvalidierung

- : Prüft, ob die Spielregeln bei Bridge-Änderung eingehalten werden.
- Beteiligte Geschäftsobjekte: Bridge, Node, Network
- Eingabe: Node1, Node2, Bridge-Typ
- Ausgabe: Validierungsstatus
- Erfolgsbedingung: Regelkonform

Prozess 8: Lösungskontrolle

- : Prüft, ob das Network vollständig gelöst ist (alle Nodes korrekt verbunden).
- Beteiligte Geschäftsobjekte: Bridge, Node, Network
- Eingabe: Aktuelles Network
- Ausgabe: Lösungsstatus
- Erfolgsbedingung: Level gelöst

2.2.3 Progression und Feedback

Prozess 9: Level erfolgreich abschließen

- : Markiert Level als gelöst und gibt Gewinn-Feedback.
- Beteiligte Geschäftsobjekte: Level, GameSession
- Eingabe: Gelöstes Level
- Ausgabe: Level als gelöst markiert
- Erfolgsbedingung: Status aktualisiert

Prozess 10: Bewertung und Scoring

- : Gelöstes Network wird bewertet und Score angezeigt/verglichen.
- Beteiligte Geschäftsobjekte: Level, GameSession, Network
- Eingabe: Gelöstes Network
- Ausgabe: Score-Feedback
- Erfolgsbedingung: Score gespeichert

Prozess 11: Fortschritt und Statistiken speichern

- : Speichert Fortschritt, Stats und Level-Freischaltung persistent.
- Beteiligte Geschäftsobjekte: Level, GameSession
- Eingabe: Level-Ergebnis
- Ausgabe: Persistenter Fortschritt
- Erfolgsbedingung: Alles gespeichert

2.2.4 Tutorial/Hilfesystem

Prozess 12: Tutorial-Ablauf für neue Spieler¹

- : Interaktives Tutorial beim ersten Spielstart.
- Beteiligte Geschäftsobjekte: GameSession
- Eingabe: Neuer Spieler
- Ausgabe: Tutorial abgeschlossen
- Erfolgsbedingung: Spieler trainiert

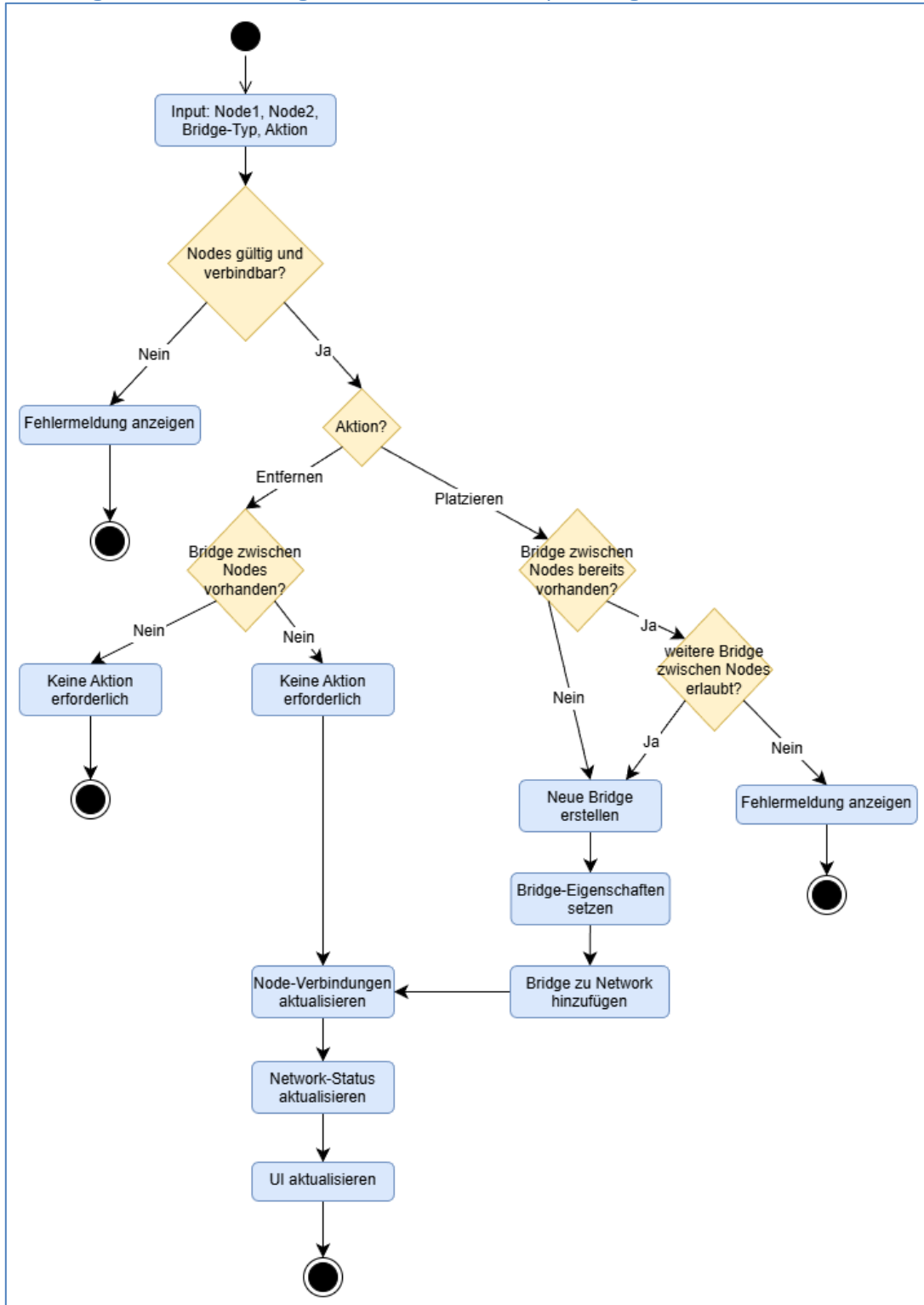
Prozess 13: Hinweis-System¹

- : Strategische Zugvorschläge für den Spieler während des Spiels.
- Beteiligte Geschäftsobjekte: Network, Nodes, Bridges
- Eingabe: Aktuelles Network
- Ausgabe: Bridge-Vorschlag
- Erfolgsbedingung: Hinweis generiert

¹ Dieser Prozess wurde in Phase 3 aufgrund des begrenzten Zeitrahmens nicht umgesetzt und stellt eine bewusst dokumentierte technische Schuld dar.

2.3 UML-Aktivitätsdiagramm eines ausgewählten Prozesses

Abbildung 2: UML Aktivitätsdiagramm Brücken entfernen/hinzufügen



Quelle: Erstellt mit draw.io

3 Geschäftsregeln

3.1 Budget- und Kostenregeln

GR-01: Jedes Level startet mit einem fest definierten Budget.

GR-02: Das verfügbare Budget darf zu keinem Zeitpunkt negativ werden.

GR-03: Jeder Bridge-Typ hat feste Kosten, die bei Platzierung sofort vom Budget abgezogen werden.

GR-04: Entfernte Bridges geben die Kosten vollständig an das Budget zurück.

3.2 Netzwerk-Topologie-Regeln

GR-05: Alle Nodes eines Levels müssen über Bridges vom Server aus erreichbar sein.

GR-06: Zwischen zwei Nodes können mehrere Bridges existieren, abhängig vom Node-Typ und dessen Verbindungskapazität.

GR-07: Ein Node kann nicht mit sich selbst verbunden werden.

3.3 Node-Spezialisierungs-Regeln

GR-08: Pro Level existiert EXAKT ein Server-Node.

GR-09: Server-Nodes MÜSSEN mindestens 2 Verbindungen haben.

GR-10: Firewall-Nodes dürfen MAXIMAL 2 Verbindungen haben.

GR-11: Ein Node kann seinen Typ während des Spiels NICHT ändern.

GR-12: Von allen Node-Typen (exklusive Server) können mehrere Instanzen pro Level existieren.

3.4 Performance- und Redundanz-Bewertungsregeln

GR-13: Die Netzwerk-Performance entspricht der durchschnittlichen effektiven Bandbreite (Bottleneck) pro Client zum Server.

GR-14: Die Netzwerk-Redundanz ist die maximale Anzahl ausfallender Brücken, bis mindestens ein Knoten vom Server getrennt wird.

3.5 Spielablauf-Regeln

GR-15: Spieler kann jederzeit gelegte Bridges entfernen und erhält die Kosten zurück.

GR-16: Level-Reset ist jederzeit möglich, setzt Budget und Netzwerk auf Anfangszustand zurück.

GR-17: Bei Regelverstoß erscheint eine Fehlermeldung und die Aktion ist nicht ausführbar.

4 Systemschnittstellen

Das Network Architect Spiel ist als eigenständige Desktop-Anwendung konzipiert und benötigt keine Systemschnittstellen zu externen Systemen. Alle Funktionen werden vollständig lokal abgewickelt:

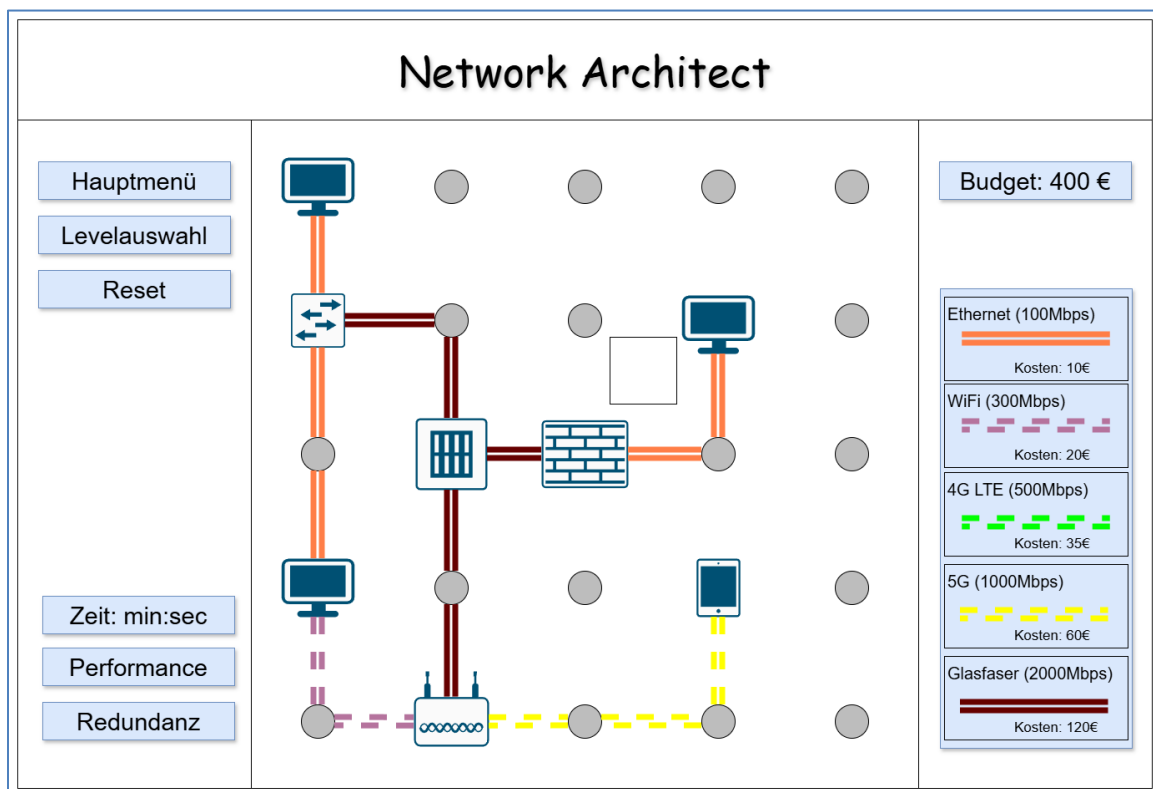
- Spiel-Logik: Interne Verarbeitung ohne externe Abhängigkeiten
- Level-Verwaltung: Statische Level-Daten im Anwendungspaket
- Score-Persistierung: Lokale SQLite-Datenbank
- Spielstand-Speicherung: Lokale SQLite-Datenbank
- Einstellungen: Lokale Konfigurationsdateien

5 Benutzerschnittstellen

5.1 Hauptspielbildschirm

Der Hauptspielbildschirm ist die zentrale Arbeitsoberfläche von Network Architect. Links befindet sich eine Seitenleiste mit den Buttons „Hauptmenü“, „Levelauswahl“ und „Reset“ sowie Statusanzeigen für Zeit, Performance und Redundanz. In der Mitte liegt das Spielfeld als Raster mit den platzierbaren Nodes (Server, Clients, Firewall, Router usw.) sowie den bestehenden und geplanten Verbindungen. Rechts werden das verfügbare Budget sowie die verfügbaren Verbindungstypen (Ethernet, WiFi, 4G LTE, 5G, Glasfaser) inklusive Bandbreite und Kosten angezeigt. Der Spieler wählt zunächst einen Verbindungstyp aus der rechten Leiste und verbindet anschließend per Klick oder Drag & Drop zwei Nodes im Spielfeld, um eine Bridge zu erzeugen. Dabei werden die in Kapitel 3 definierten Geschäftsregeln (z.B. Budgetgrenzen und Verbindungskapazitäten) geprüft; bei gültigen Aktionen wird die Verbindung eingezeichnet und das Budget angepasst.

Abbildung 3: GUI Hauptspielbildschirm

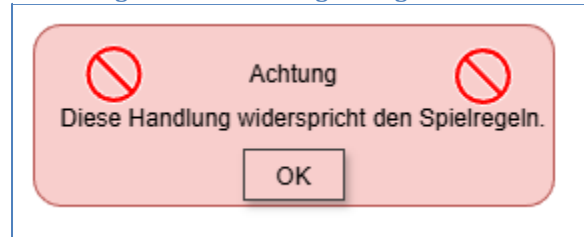


Quelle: Erstellt mit draw.io

5.2 Fehlermeldungsdialog bei Regelverstoß

Bei ungültigen Aktionen, z.B. wenn ein Node mit sich selbst verbunden werden soll, das Budget nicht ausreicht oder eine Verbindung gegen die festgelegten Topologie- oder Spezialisierungsregeln verstößt, erscheint ein Fehlermeldungsdialog. Dieser Dialog informiert den Spieler darüber, dass die Handlung den Spielregeln widerspricht, und bietet einen „OK“-Button zum Schließen. Die zugrunde liegende Aktion wird nicht

Abbildung 4: Fehlermeldungsdialog



Quelle: Erstellt mit draw.io

ausgeführt, sodass der Spielzustand konsistent bleibt

5.3 Levelabschluss-Dialog

Nach erfolgreichem Abschluss eines Levels – d.h. alle Nodes sind vom Server aus erreichbar und die Abschlussbedingungen des Levels sind erfüllt – wird ein Levelabschluss-Dialog angezeigt. Dieser Dialog zeigt eine kurze Erfolgsmeldung sowie die wichtigsten Kennzahlen: benötigte Zeit, Performance-Score und Redundanz-Score. Der

Abbildung 5: Levelabschluss-Dialog



Quelle: Erstellt mit draw.io

Spieler bestätigt den Dialog mit „OK“ und gelangt anschließend zum nächsten Schritt, beispielsweise zum nächsten Level oder zurück zur Levelauswahl.

6 Abbildungsverzeichnis

Abbildung 1: UML-Diagramm, Beziehungen der Geschäftsobjekte.....	5
Abbildung 2: UML Aktivitätsdiagramm Brücken entfernen/hinzufügen.....	10
Abbildung 3: GUI Hauptspielbildschirm	13
Abbildung 4: Fehlermeldungsdialog.....	14
Abbildung 5: Levelabschluss-Dialog.....	14